

TEMA: Django

Resultados de Aprendizaje:

- Que los estudiantes pueden crear su primer proyecto con Django
- Que los estudiantes puedan crear aplicaciones en Django
- Que los estudiantes puedan pasar datos desde las vistas a los témpate

DESARROLLO

Instalacion de Django

- Debera de crera una carpeta de nombre **proyectos_django**
- Debera de crear el entorno virtual con el comando **> python -m venv .venv**
- Activa el entorno virtual **> .\.venv\Scripts\activate** ,para linux
- Para platadorma como windows puede activar en entorno virtual con el comando **> .\.venv\Scripts\activate.bat** , simpre y cuando su terminar sea CMD
- Para pode instlar Django lo podemos hace con el comando **pip install django**
- Tambien puede instalar una version predeterminada de Django con el comando , **pip install django==6.1**

Creacion de proyecto

- Una vez tenga instalado djando , este nos ha agergado un nuevo comando a la consola que es **> django-admin** , que al ejecutarlo nos mostrar toda una seria de comando nuevos que podriamos ocupar

```
> django-admin
Type 'django-admin help <subcommand>' for help on a specific subcommand.
Available subcommands:
[django]
  check
  compilemessages
  createtables
  dbshell
  diffsettings
  dumpdata
  flush
  inspectdb
  loaddata
  makemessages
  makemigrations
  migrate
  optimizemigration
  runserver
  sendtestemail
  shell
  showmigrations
  sqlflush
  sqlmigrate
  sqlsequencereset
  squashmigrations
  startapp
  startproject
  test
  testserver
Note that only Django core commands are listed as settings are not properly configured (error: Requested setting INSTALLED_APPS, but settings are not configured. You must either define the environment variable DJANGO_SETTINGS_MODULE or call settings.configure() before accessing settings.).
```

- El siguiente paso es la creacion del poyecto este es quien gestionar toda la confiuracion de la aplicasion , para la realizacion de esto lo puede realizar con el comando **> django-admin startproject comercial** , donde comercial sera el nombre del proyecto , evite utilizar (-) con el nombre del proyecto porque esto lanzara error puede utilizar (_) si lo prefiere

9. Lo anterioro generar una capeta llamada **comercial** , debera de navegar a dicho directorio con el comando **> cd comercial**

10. Generando el sigueinte sistema de directorio:

```
comercial/  
  manage.py  
  comercial/  
    __init__.py  
    settings.py  
    urls.py  
    asgi.py  
    wsgi.py
```

11. Archivos generados y que hacen:

manage.py: Una utilidad de la línea de comandos que le permite interactuar con este proyecto Django de diferentes formas.

comercial/: Un directorio que es el paquete Python . Su nombre es el nombre del paquete Python que necesitarás usar para importar cualquier cosa dentro de él proyecto.

comercial/__init__.py: Un archivo vacío que le indica a Python que este directorio debe considerarse un paquete de Python.

comercial/settings.py: Configuraciones/configuración para este Django proyecto. La configuración de Django le informará todo sobre cómo funciona la configuración trabajar.

comercial/urls.py: Las declaraciones URL para este proyecto Django; una «tabla de contenidos» de su sitio basado en Django.

comercial/asgi.py: Un punto de entrada para servidores web compatibles con ASGI Sirve a tu proyecto.

comercial/wsgi.py: Un punto de entrada para que los servidores web compatibles con WSGI puedan servir su proyecto.

12. Agregue el archivo **requirements.txt** en el directorio principal del proyecto

```
Comercial/  
  manage.py  
  requirements.txt  
  comercial/  
    __init__.py  
    settings.py  
    urls.py  
    asgi.py  
    wsgi.py
```

Ejecute el comando **> pip freeze** , este le mostrar el paquete instalado y su version para que usted lo pued agergar al archivo **requirements.txt** , recuerde realizar este comando cada que instale un nuevo paquete que necesite en produccion

```
Django==6.1.1
```

13. Para poder ver que comados tiene asociado el archivo **manage.py** puede ejecutar el comadno **> python manage.py** , para mostrar todo los comando validos que tiene asociado el proyecto , entre los que puede encontrar

```
> python .\manage.py

Type 'manage.py help <subcommand>' for help on a specific subcommand.

Available subcommands:

[auth]
  changepassword
  createsuperuser

[contenttypes]
  remove_stale_contenttypes

[django]
  check
  compilemessages
  createcachetable
  dbshell
  diffsettings
  dumpdata
  flush
  inspectdb
  loaddata
  makemessages
  makemigrations
  migrate
  optimizemigration
  sendtestemail
  shell
  showmigrations
  sqlflush
  sqlmigrate
  sqlsequencereset
  squashmigrations
  startapp
  startproject
  test
  testserver

[sessions]
  clearsessions

[staticfiles]
  collectstatic
  findstatic
  runserver
```

Ejecucion del proyecto

14. Para la ejecusaion del poryecto lo puede ralizar con el comando **> python .\manage.py runserver**
15. Ontenido un resultado como el siguiente , y proporcionado una direccion web para poder visualizar nuestro proyecto , en la direccion web <http://127.0.0.1:8000>

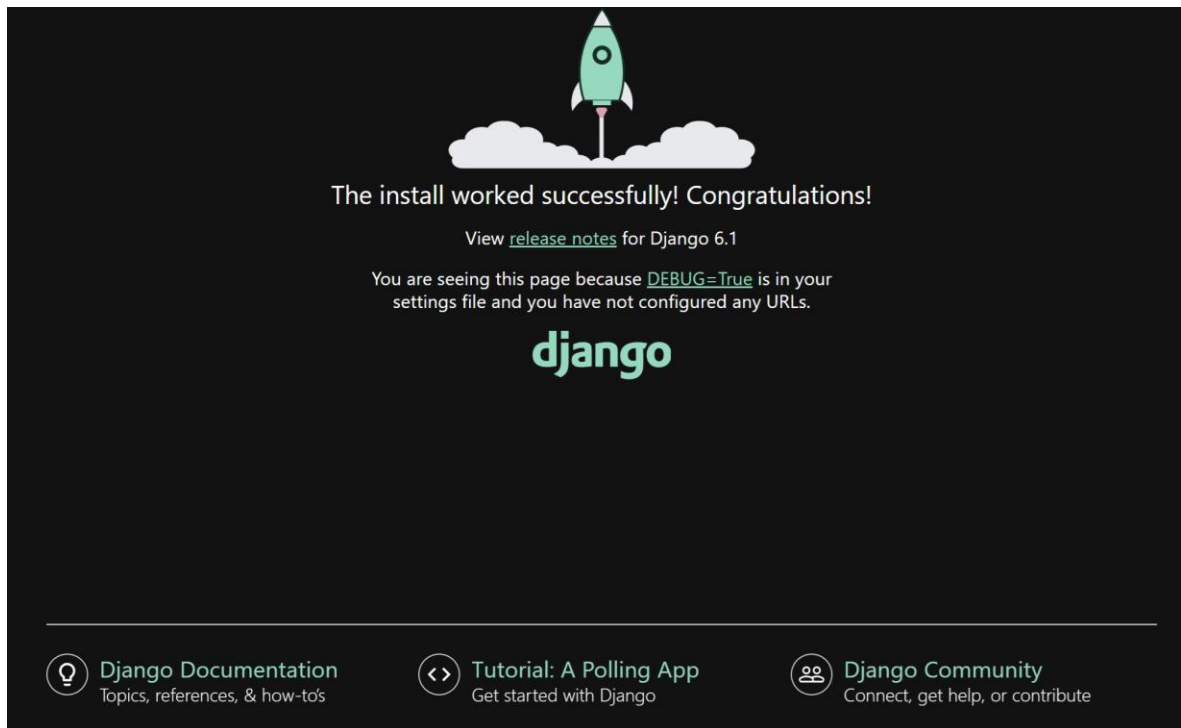
```
> python .\manage.py runserver
Watching for file changes with StatReloader
Performing system checks ...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s)
: admin, auth, contenttypes, sessions
Run 'python manage.py migrate' to apply them.
September 09, 2026 - 16:10:30
Django version 6.1, using settings 'comercial.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI serv
er instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/howto/deployment/
```

16. Mostando una pagina web definida por defecto



MODELO MVT

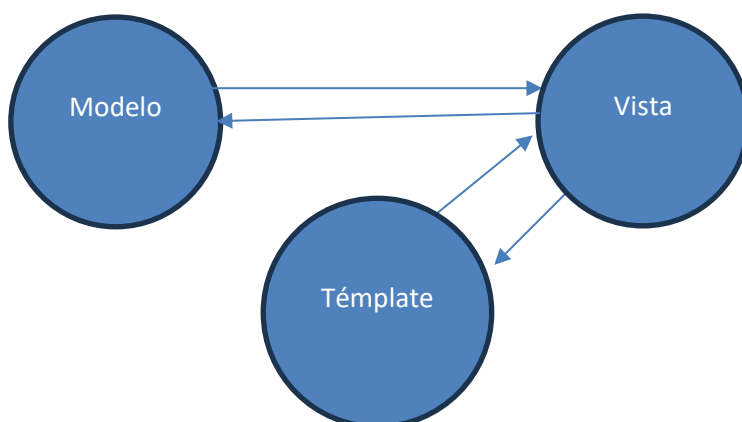
Estas nos familiarizado a trabajar con el modelo de datos MVC (modelo , vista , controlador) , djando a cambianod un poco este paradigma porque cuando trabajamos con el debemos de trabajar con MVT

- M : modelo
- V : Vista
- T : Template

Que seria el modelo , para comprender esto tenemos que pensar que el modelo seria la parte de los datos , como guarda los datos , como los procesa donde esta la logia del negocio

Que seria la vista , para comprender esto tenemos que pensar que la vista es un conector quien tendra acceso a los datos y tendra que decidir a donde se dirigen los datos

Que seria el Template , el template seria nuestra parte grafica de como mostramos la inforamcion ese html , css que utilizamos para mostrar la informacion .



Los modelos le pasaran informacion a las vista las vista a los template y desde el template se puede solicitar informacion a la vista la vista hace peticion al modelo y todo comienza de nuevo

Nota : nunca deberiamos conectar un modelo directo al template aun que esto se pueda

Creacion de app

17. Para poder visualizar el patron MVT , django nos provee la posibilidad de agregar app a nuestro proyecto y es encada app que maneja el patron MVT

18. Para ellos utilizaremos la terminal y nos posicionaremos en nuestro proyecto y utilizaremos el comando **> python .\manage.py startapp catalogo**

```
> python .\manage.py startapp catalogo
```

19. Lo anterior genera un directorio como el siguiente

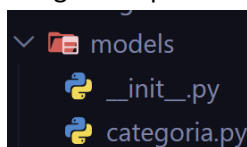
```
catalogo/  
  __init__.py  
  admin.py  
  apps.py  
  migrations/  
    __init__.py  
  models.py  
  tests.py  
  views.py
```

20. Hoy crearemos una nueva carpeta llamada models , donde agregaremos todos nuestros modelos de la base de datos que necesitemos , esto porque necesitamos crear mas de un modelo , si solo necesitaramos una tabla en la base de datos bastara con el archivo de models ,

Debera de eliminar el archivo models.py , para evitar conflictos de importacion

```
catalogo/  
  __init__.py  
  admin.py  
  apps.py  
  migrations/  
    __init__.py  
  models/  
  tests.py  
  views.py
```

21. El siguiente paso es agregar los archivos **__init__.py** y **categoria.py** dentro de la carpeta models



```
models  
  __init__.py  
  categoria.py
```

22. Hoy crearemos nuestro primer modelo, tenga en cuenta que este modelo representará los datos de nuestra base de datos para ello modificamos el archivo `> catalogo/models/categoria.py`, de la siguiente manera

```
from django.db import models

# Create your models here.

class Categoria(models.Model):
    """# define un campo de tipo texto con máximo de 255 caracteres"""
    nombre = models.TextField(max_length=255, verbose_name="Nombre")
    """# define un campo de tipo texto que puede ser nulo con máximo de 255 caracteres"""
    descripcion = models.TextField(
        max_length=255, null=True, verbose_name="Descripción")
    """# define campos de tipo fecha y hora para almacenar la fecha de creación y actualización del registro"""
    created_at = models.DateTimeField(verbose_name="Fecha de creación")
    updated_at = models.DateTimeField(verbose_name="Fecha de actualización")

    """# function que mostrar la information de la tabla en consola"""
    def __str__(self):
        return f'id: {self.id} Nombre: {self.nombre} Description: {self.descripcion} Fecha de creación : {self.created_at} Fecha de actualización {self.updated_at}"
```

23. El siguiente paso es modificar el archivo `> catalogo/models/__init__.py`, y agregar la importación del modelo que se transforma en la tabla en la base de datos, si creamos más modelos todo deberán ser importados a estos archivos

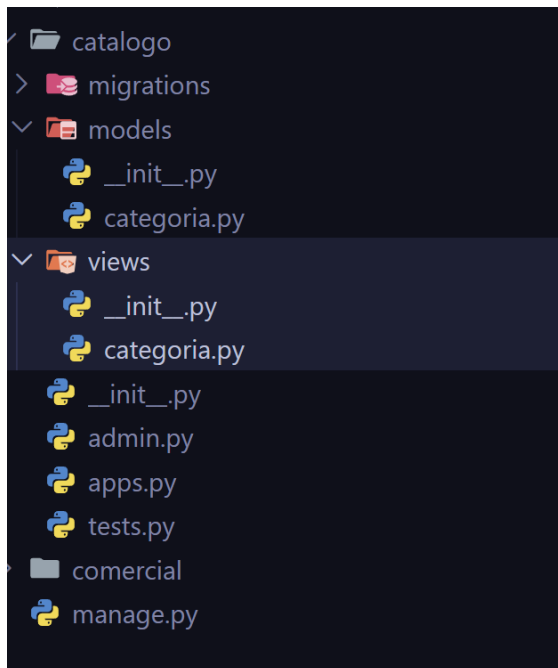
```
from .categoria import Categoria
```

Creación de vistas

24. El siguiente paso que haremos es definir las vistas como tendremos muchas seguiremos los mismos pasos que con los modelos, crearemos una clase llamada `views` y eliminaremos el archivo `views.py`, también agregaremos los archivos `__init__.py` y `categoria.py`

```
catalogo/
  __init__.py
  admin.py
  apps.py
  migrations/
    __init__.py
  models/
  tests.py
  views/
    __init__.py
    categoria.py
```

25. Si ha seguido los pasos anteriores debería de tener un directorio como el siguiente



26. El siguiente paso que aremos es definir la vista de categoria modificando el archivos **views/categoria.py**

```
from django.shortcuts import render

# Create your views here.

def lista_categorias(request):
    return render(request, "categoria/lista.html")
```

Muy importante la funcion **render** , lo que realiza es renderizar el template **lista.html** , este archivo aun no existe pero lo crearemos en los siguientes pasos

Creacion de templates

27. Hoy crearemos una nuevos directorios dentro de la app categorias

Templates/categoria/lista.html ,

Quedando el arbol de directoio de la app de la siguiente manera

catelogo/

__init__.py

admin.py

apps.py

templates/

categoria/

lista.html

migrations/

 __init__.py

Models/

tests.py

views/

28. Modificamos de manera preliminar nuestro template , **lista.html** de la siguiente manera

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Categorias</title>
  </head>
  <body>
    <h1>Lista de Categorias</h1>
  </body>
</html>
```

29. Ya casi hemos terminado de definir nuestra app , pero aun falta configurar como podremos acceder a nuestras vista , para ello crearemos otro archivo mas en nuestra app llamado , **urls.py** de modo que el directorio de la app y quedara de la siguiente manera

catalogo/
 __init__.py
 admin.py
 apps.py
 urls.py
 templates/
 migrations/
 models/
 tests.py
 views/

30. Modificamos el archivo **urls.py** para que quede de la siguiente manera , de esta manera estaremos sirviendo la vista

```
from django.urls import path
from .views.categoria import lista_categorias

urlpatterns = [
    path("categoria/", lista_categorias, name="list_categorias")
]
```

Notas : from .views.categoria import lista_categorias , cuidado con el punto al inicio

31. Pero aun no hemos terminado , nos queda dar de alta la app dentro del proyecto , comercial
32. Para dar de alta la app modificaremos el archivo **comercial/settings.py** y agregaremos al app para que el proyecto este pendiente de sus urls y de sus migraciones en la base de datos ,

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'catalogo'
]
```

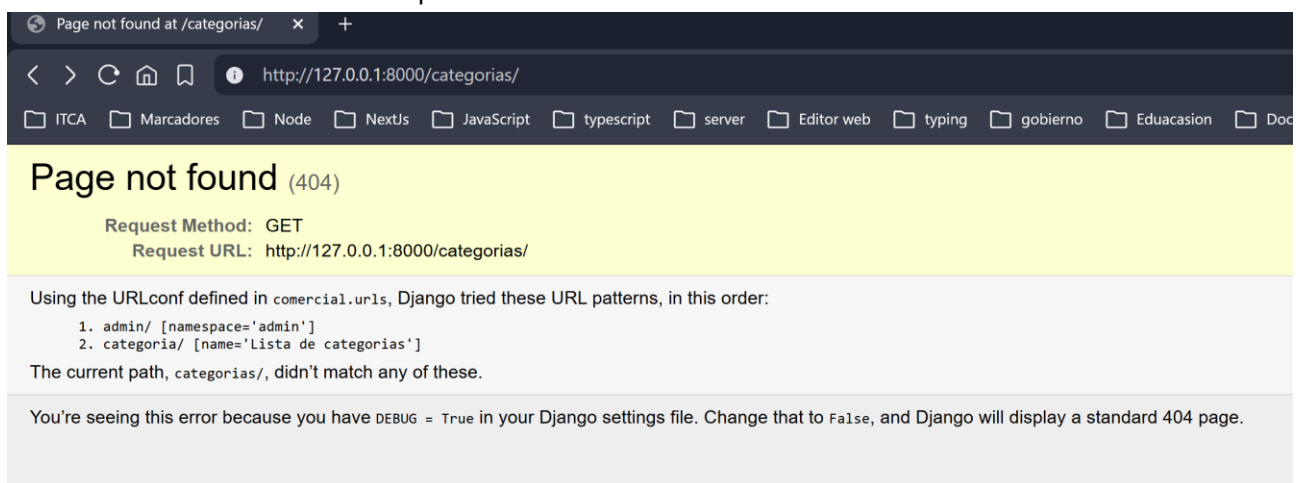
33. Como ultimo paso deberemos dar de alta las url en el proyecto , para ello deberemos de modificar el archivo **comercial/urls.py** ,

```
from django.contrib import admin
from django.urls import path include

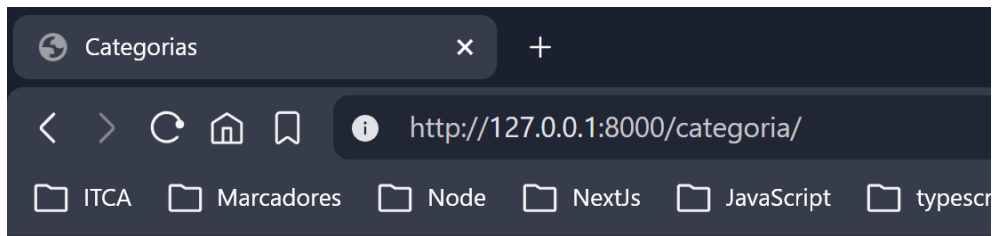
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('catalogo.urls'))
]
```

Nota : cuando realice el include , catalogo es el nombre de al app y urls es el nombre del archivo con las urls de la app de categoriras

34. Hoy estamos listos para probar nuestra proyecto y lo aremos con el comando ➤ **python .\manage.py runserver**
35. obtendra algo muy difente de que obtuvimos la primera vez , esta vez obtendremos una pagina donde le mostrara todas las url disponibles



36. si se dirige la url de categorias <http://127.0.0.1:8000/categoria/> podra observar lo siguiente



Lista de Categorías

Envío de datos desde la vista al template:

37. aun que hemos creado el modelo de categorias para la creacion de los datos en la base de datos , aun no esta generada , dejaremos eso para la siguiente sesiones , pero hoy aprenderemos como pasar los datos desde la vista al template
38. para ello modificaremos el archivo **> categorias/view.py**

```
from django.shortcuts import render

# Create your views here.

def lista_categorias(request):
    lista_categorias = [
        {
            'id': 1,
            'nombre': 'tv',
            'descripcion': 'Televisores de alta gama'
        },
        {
            'id': 2,
            'nombre': 'Laptop',
            'descripcion': 'Computadoras portátiles'
        },
    ]
    return render(request, "categoria/lista.html", context={
        'categorias': lista_categorias
    })
```

Denote que estamos generando una lista de categorias que son enviadas al template gracias al context , este genera un diccionario de datos que seran enviado , y **pueden ser mas de un dato**

39. hoy modificaremos nuestro template , **templates/categoria/lista.html**

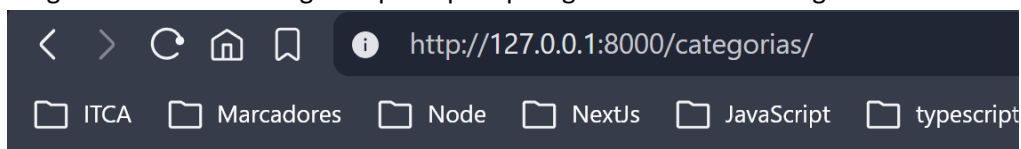
```

<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Categorias</title>
  </head>
  <body>
    <h1>Lista de Categorias</h1>
    <ul>
      {% for categoria in categorias %}
      <li>{{categoria.id}} {{categoria.nombre}}</li>
      {% endfor %}
    </ul>
  </body>
</html>

```

Nota : denote que podemos utilizar ciclicas de tipo **for** utilizandod **{% %}** y para poder imprimir en pantalla utilizamos los simpoblos **{{ }}** que hace un simil a php **<?=>**

40. hoy ya podemos probar nuestra aplicassion nuevamente ➤ **python .\manage.py runserver** y dirigirnos a la url de categorias para que optengamos el resultado siguiente



Lista de Categorias

- 1 tv
- 2 laptop

41.

	EJERCICIOS DE PRÁCTICA:
---	--------------------------------